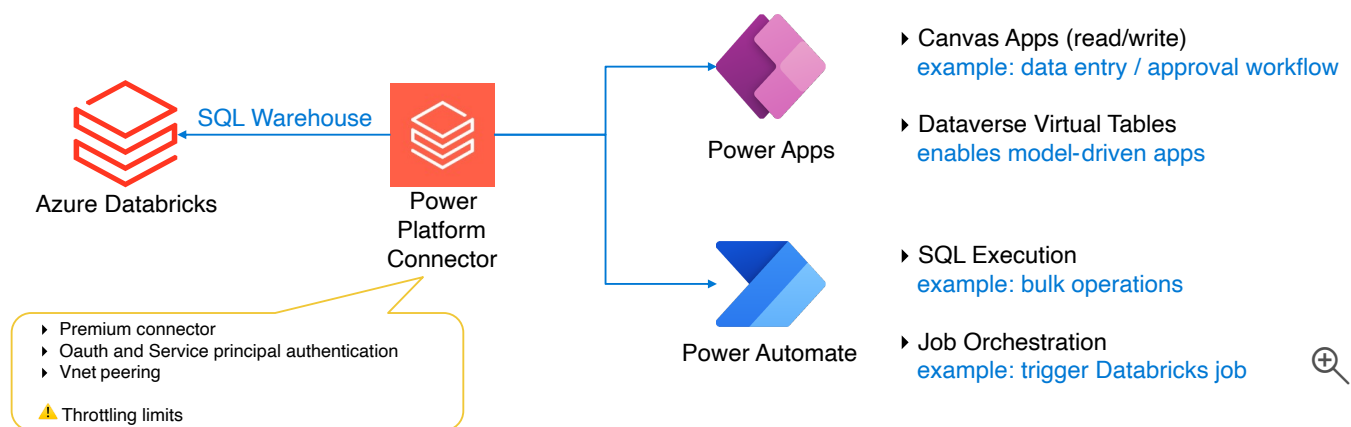


Understand integration with Power Platform

13 minutes

Power Platform enables you to build custom applications, automate workflows, and analyze data without extensive coding. Integrating Azure Databricks with Power Platform allows you to bring governed, high-quality data from your lakehouse directly into business applications and automation workflows. This integration supports data engineers in delivering data solutions that empower business users across the organization.

The integration between Azure Databricks and Power Platform works through the **Azure Databricks connector**. This connector enables Power Apps and Power Automate to access data stored in Unity Catalog while preserving your governance controls. You can build applications that read and write data, automate job execution, or orchestrate complex data workflows—all while maintaining the security policies and access controls defined in Unity Catalog.



Connect Power Platform to Azure Databricks

Power Platform connects to Azure Databricks through a **premium connector** that requires specific prerequisites. You need a premium Power Apps license, a Microsoft Entra ID account, and access to a SQL warehouse in your Azure Databricks workspace. The connector uses SQL warehouses as

the compute resource because they provide optimized query performance and serverless scaling for analytical workloads.

To establish a connection, you create an **Azure Databricks connection** in Power Apps or Power Automate. From the Connections page, you search for the Azure Databricks connector and select your authentication method. The connector supports two authentication types: **OAuth connection** (which uses Microsoft Entra ID to authenticate individual users) and **service principal connection** (which uses fixed credentials for authentication). Each authentication type serves different scenarios and offers different capabilities.

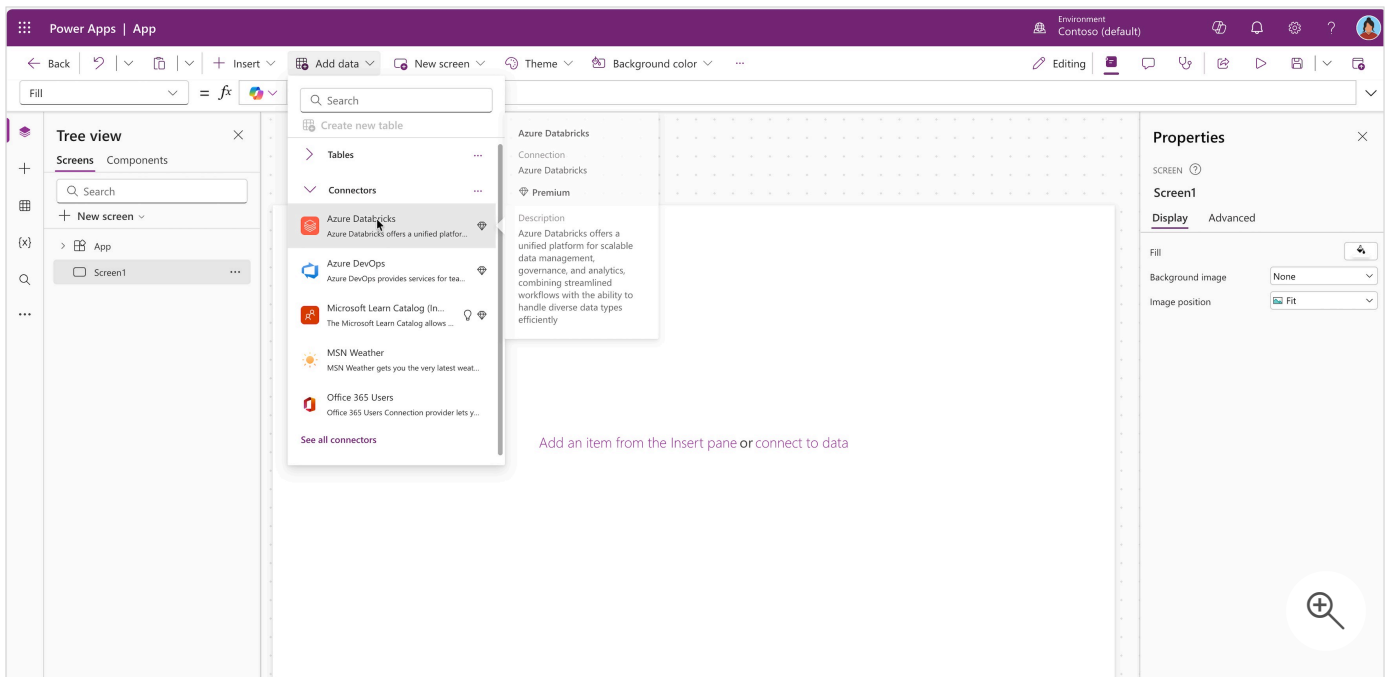
To complete the connection setup, you provide the **Server Hostname** and **HTTP Path** from your SQL warehouse. These connection details identify which compute resource Power Platform uses to query your data. You can find these values in the connection details section of your SQL warehouse in the Azure Databricks workspace.

Note

If your Azure Databricks workspace uses virtual networks, you can integrate Power Platform using VNet peering for private connectivity or by configuring IP access lists to allow AzureConnectors IP ranges.

Build applications with Power Apps

Power Apps enables you to build **canvas apps** that provide custom interfaces for reading and writing data. When you connect a canvas app to Azure Databricks, you select a catalog and choose the tables that your application will access. The app can perform create, update, and delete operations on your data, making it possible to build data entry forms, approval workflows, or dashboards that interact directly with your lakehouse.



With a direct connection to Azure Databricks, you can build canvas apps that interact with data while **preserving Unity Catalog governance**. When you use OAuth authentication, Unity Catalog evaluates each user's permissions in real time, ensuring that the app respects row-level security, column masking, and other access controls. This approach lets business users work with trusted data through an intuitive interface without compromising security.

For applications that require **bulk operations**, Power Automate flows can process multiple changes in a batch. This pattern handles large-scale updates more efficiently than individual operations from the canvas app.

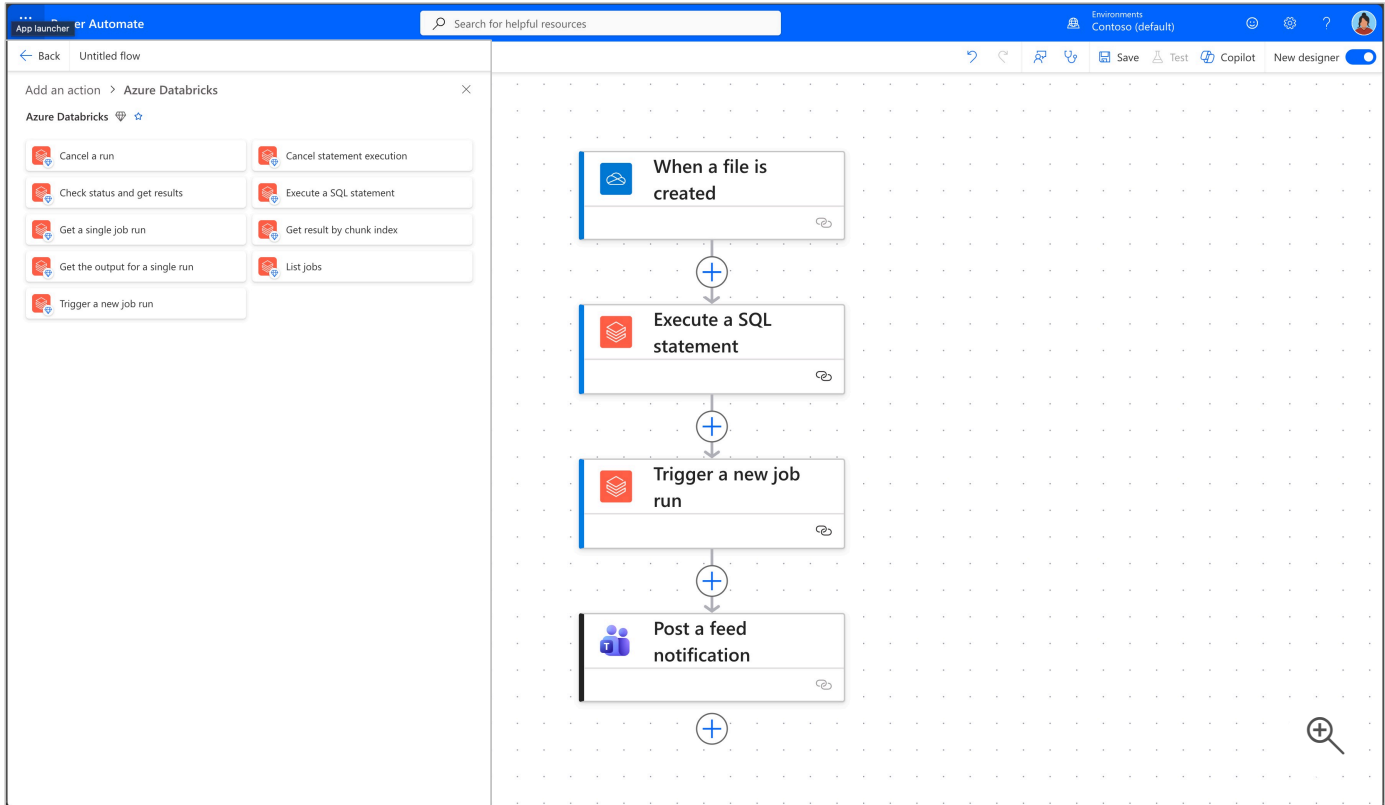
Power Apps also supports **Dataverse virtual tables** backed by Azure Databricks data. Virtual tables integrate data from Azure Databricks with Microsoft Dataverse without physically copying the data. This approach enables you to build **model-driven apps** that provide structured forms, views, and business logic on top of your Databricks data.

Tip

For better performance, use direct connections rather than virtual tables when possible. While virtual tables don't consume Dataverse storage capacity, direct connections typically offer faster query response times.

Automate workflows with Power Automate

Power Automate enables you to build **flows** that automate business processes and data operations. The Azure Databricks connector exposes two primary capabilities within Power Automate: **SQL statement execution** for running queries and **job orchestration** for triggering existing Databricks jobs. These capabilities enable you to create automated workflows that respond to business events, schedule data processing tasks, or coordinate complex data pipelines.



With **SQL statement execution**, you can write and execute SQL queries directly from a flow. You create a flow with any trigger type—such as a schedule, a manual trigger, or an event from another service—then add an Azure Databricks action to execute SQL. The flow can retrieve query results and use them in subsequent steps, with support for handling large result sets through chunked retrieval.

Beyond SQL execution, **job orchestration** lets you trigger existing Azure Databricks jobs from Power Automate. You can start a job run, track its progress, and retrieve metadata about the run including its status, start and end times, and execution duration. This integration enables you to **coordinate data processing workflows** with business processes—for example, triggering a data refresh when new files arrive or sending notifications when a data quality job completes.

The connector also supports canceling running statements or jobs, listing available jobs, and retrieving output from completed runs. These capabilities enable scenarios like automatic retry

logic, conditional workflows based on data quality checks, or multi-step data pipelines that combine Databricks processing with other services.

Important

The Power Platform connector has throttling limits. Design your flows to batch operations appropriately when processing large volumes.

Integration scenarios

The Azure Databricks and Power Platform integration enables several practical scenarios that bridge the gap between data engineering and business operations. Consider a **planning and forecasting application** where business users submit next month's forecasts, adjust revenue targets, or update budget allocations through a Power App that writes directly to a Unity Catalog table. With OAuth authentication and Unity Catalog governance, each user sees only the data they're authorized to access, and all changes are tracked in the table's audit log.

Another scenario involves **automated data quality workflows**. You build a Power Automate flow that runs on a schedule, executes a SQL query to check data quality metrics, and sends notifications to the data team when quality thresholds are exceeded. The flow can also trigger a Databricks job to repair data issues automatically or escalate to a human review process based on the severity of the issue.

For **event-driven data processing**, you create a flow that monitors a business system for events—such as a new customer order or a completed transaction—and triggers a Databricks job to process the associated data. The flow passes relevant parameters to the job, waits for completion, and updates the business system with the results. This pattern enables real-time data integration without building custom integration code.

Dashboard refresh automation represents another valuable use case. You schedule a Power Automate flow that executes SQL to prepare aggregated data, stores the results in a table, and triggers a refresh of a business intelligence dashboard. This approach provides business users with up-to-date insights without manual intervention from the data team.

These scenarios become possible because the integration **preserves governance while enabling access**. Unity Catalog continues to enforce all security policies, data masking, and access controls

even when data flows through Power Platform. Business users gain the ability to work with data through familiar interfaces while data engineers maintain centralized control over data quality, security, and compliance.

Considerations and limitations

While the Azure Databricks Power Platform connector provides broad capabilities, some considerations affect how you design your solutions. The connector **doesn't support government clouds**, including US Government and China Cloud environments. Organizations operating in these environments need alternative integration approaches.

Within Power Apps, certain **PowerFx formulas** calculate values using only data that has been retrieved locally to the app, rather than delegating computation to Azure Databricks. For large datasets, this can affect performance and accuracy. Consider filtering data or performing aggregations in SQL within Azure Databricks before bringing data into Power Apps.

Concurrent write operations from multiple Power Apps users or flows benefit from row-level concurrency features in Azure Databricks. These features are available in recent Databricks Runtime versions and help reduce write conflicts when multiple users modify data simultaneously.

When choosing between **virtual tables and direct connections**, consider the trade-offs. Virtual tables enable model-driven apps and integrate with the broader Dataverse ecosystem, but they don't support OAuth credential passthrough. Direct connections offer better performance and support OAuth for governance passthrough, but they only work with canvas apps. Choose the approach that best fits your application architecture and governance requirements.

Finally, **data policies** in Power Platform affect connector usage. You can add the Azure Databricks connector to a Business data policy to control which other connectors can share data with it, helping you maintain compliance with data handling regulations.

Next unit: Understand integration with Copilot Studio

[< Previous](#)[Next >](#)